Security Audit Report

Report ID: MX-VM-V1.4-AUDIT-001
Project: mx-chain-vm-v1_4-go

1. Executive Summary

This report documents critical resource management vulnerabilities identified in the WebAssembly (WASM) host implementation of the `mx-chain-vm-v1_4-go` project. These vulnerabilities allow for host-level Resource Exhaustion (DoS) by bypassing established gas-metering mechanisms.

2. Vulnerability Details

2.1 [REAL-4] Unbounded BigInt Shift Left (Host OOM)

Severity	Critical
Location	vmhost/vmhooks/bigIntOps.go:v1_4_bigIntShl
Impact	Host Process Termination (OOM)

Technical Analysis: The `v1_4_bigIntShl` hook accepts an `int32` shift amount. Providing `MaxInt32` causes `math/big` to allocate a ~256MB buffer before any gas is charged for the final result size.

```
// Location: vmhost/vmhooks/bigIntOps.go
func v1_4_bigIntShl(context unsafe.Pointer, destinationHandle,
opHandle int32, bits int32) {
    // ... base gas ...
    dest.Lsh(a, uint(bits)) // Unbounded allocation
```

```
    managedType.ConsumeGasForBigIntCopy(dest)
}
```

2.2 [REAL-6] Unmetered Large Memory Allocation (OOM/DoS)

Severity	High
Location	vmhost/vmhooks/baseOps.go:getArgumentsFromMemory
Impact	Host Memory Exhaustion

Technical Analysis: The helper function `getArgumentsFromMemory` performs multiple `MemLoad` operations that allocate host-side Go slices before gas is deducted in the calling function. This allows an attacker to OOM the host for free.

```
// Location: vmhost/vmhooks/baseOps.go
func getArgumentsFromMemory(host vmhost.VMHost, numArguments
int32, ...) {
    // ...
    argumentsLengthData, err :=
runtime.MemLoad(argumentsLengthOffset, numArguments*4)
    // ...
    data, err := runtime.MemLoadMultiple(dataOffset,
argumentLengths)
    // ...
    return data, totalArgumentBytes, nil
}
```

3. Remediation Strategy

3.1 REAL-4: Implement a hard cap on `bits` and perform gas deduction *before* the shift.

3.2 REAL-6: Implement "Gas-First" loading by deducting gas for metadata and argument slices *before* allocation.